

CorpusMind

User Guide

Version 0.7.0 | Pre-Release | AGPL-3.0-only

Local-first, AI-native research environment for
corpus linguistics and multimodal discourse analysis

97 tests passing | 25 AI tools | 85 API routes | 12 frameworks | 20
statistical formulas

Table of Contents

Table of Contents

1. Installation
 2. Starting the Engine and Web App
 3. Creating a Project and Uploading Texts
 4. Concordance Search (KWIC)
 5. Frequency Analysis
 6. Collocation Analysis
 7. Keyness Analysis
 8. Dispersion Analysis
 9. N-grams and Lexical Bundles
 10. POS, Grammar, and Dependency Analysis
 11. Discourse and Metadiscourse Analysis
 12. Vocabulary Profiling
 13. Sentiment Analysis
 14. Metaphor Detection
 15. Arabic Analysis
 16. Image Ingestion and Vision Analysis
 17. Visual Grammar (Kress and van Leeuwen)
 18. Multimodal Image-Text Alignment
 19. Multimodal Discourse Analysis
 20. The AI Assistant
 21. Exporting Results
 22. Saved Searches, Bookmarks, and Favorites
 23. Project Sharing and Collaboration
 24. Arabic-Specific Features Deep Dive
 25. Bilingual Corpus Tools
 26. Facial Analysis (Opt-In)
 27. At-Rest Encryption
 28. Self-Hosted Lab Deployment
 29. Accessibility Features
 30. Troubleshooting
- Getting Help

Table of Contents

1. Installation
2. Starting the Engine and Web App
3. Creating a Project and Uploading Texts
4. Concordance Search (KWIC)
5. Frequency Analysis
6. Collocation Analysis
7. Keyness Analysis
8. Dispersion Analysis
9. N-grams and Lexical Bundles
10. POS, Grammar, and Dependency Analysis
11. Discourse and Metadiscourse Analysis
12. Vocabulary Profiling
13. Sentiment Analysis
14. Metaphor Detection
15. Arabic Analysis
16. Image Ingestion and Vision Analysis
17. Visual Grammar (Kress and van Leeuwen)
18. Multimodal Image-Text Alignment
19. Multimodal Discourse Analysis
20. The AI Assistant
21. Exporting Results
22. Saved Searches, Bookmarks, and Favorites
23. Project Sharing and Collaboration
24. Arabic-Specific Features Deep Dive
25. Bilingual Corpus Tools
26. Facial Analysis (Opt-In)
27. At-Rest Encryption
28. Self-Hosted Lab Deployment
29. Accessibility Features
30. Troubleshooting

1. Installation

Prerequisites

You need three things installed on your computer:

Python 3.12 or newer. Download it from python.org. On macOS, you can also use Homebrew: `brew install python@3.12`.

Node.js 20 or newer. Download it from nodejs.org. On macOS: `brew install node@20`.

Ollama (for the AI Assistant to work locally). Download it from ollama.com. After installing, open a terminal and run:

```
ollama pull llama3.2:3b
```

This downloads a small language model (about 2 GB) that fits on any Mac. If you have 16 GB or more of RAM and want better answers, you can instead run `ollama pull llama3.1:8b`.

Clone the Repository

Open a terminal and run:

```
git clone https://github.com/waleedmandour/CorpusMind.git cd CorpusMind
```

Set Up the Engine

```
cd engine python3 -m venv .venv source .venv/bin/activate pip install -e ".[dev]"
```

On Windows, use `.venv\Scripts\activate` instead of `source .venv/bin/activate`.

Set Up the Web Frontend

Open a new terminal window:

```
cd CorpusMind/web npm install
```

Optional: Arabic Support

If you will be working with Arabic text, install the CAMeL Tools data:

```
cd engine source .venv/bin/activate pip install camel-tools pyrsistent muddler cachetools emoji future regex camel_data -i morphology-db-msa-r13 camel_data -i dialectid-model6 python -m spacy download en_core_web_sm
```

Optional: Desktop App

If you want the native desktop app instead of the browser-based PWA:

```
brew install rustup-init rustup-init cd CorpusMind/desktop/src-tauri cargo tauri dev
```

The first build takes 5 to 10 minutes. Subsequent launches are instant.

2. Starting the Engine and Web App

You need two terminal windows running.

Terminal 1: Start Ollama

```
ollama serve
```

Keep this running. It serves the local language model on port 11434.

Terminal 2: Start the engine

```
cd CorpusMind/engine source .venv/bin/activate corpusmind-engine
```

You should see:

```
Uvicorn running on http://127.0.0.1:8765
```

Terminal 3: Start the web app

```
cd CorpusMind/web npm run dev
```

Open your browser to <http://localhost:5173>.

Verify Everything Is Connected

Click the **Settings** tab in the ribbon. You should see:

- bullet Engine status: ok
- bullet Version: 0.7.0
- bullet Ollama: connected (green dot)

If Ollama shows "offline", make sure `ollama serve` is running in a terminal.

3. Creating a Project and Uploading Texts

1. Click the **Text Suite** tab in the ribbon.
2. Click the **Manage** sub-tab.
3. Click + **New** under the Projects column.
4. Enter a name (for example, "My Research") and select a language.
5. Click **Create**.
6. Click your new project to select it.
7. Click + **New** under the Corpora column to create a corpus within the project.
8. Select your corpus.
9. Drag and drop text files into the dropzone, or click to choose files.

Supported formats: TXT, DOCX, PDF, HTML, XML, CSV, Markdown.

The engine automatically detects encoding, cleans the text, tokenizes, tags with POS, lemmatizes, and parses dependencies using spaCy. You will see the document count and token count update in

real time.

Pipeline Recipe

Click the small arrow next to a corpus name to see its pipeline recipe. This shows exactly which spaCy model and version produced the annotations. This information is included when you export a Methods Section PDF, so your analysis is always reproducible.

4. Concordance Search (KWIC)

1. Select a corpus.
2. Click the **Concordance** sub-tab.
3. Type a search query in the search box.
4. Choose the search level:

bullet **word:** matches the exact surface form

bullet **lemma:** matches all inflected forms (for example, "fox" finds "fox", "foxes", "Foxes")

bullet **pos:** matches a POS tag (for example, "NOUN" finds all nouns)

1. Set the context window (how many words to show left and right of the match).
2. Click **Search**.

Results appear in a KWIC table with color-coded POS tags. Each line has a stable line ID (for example, `doc:0:3`) that the AI Assistant can cite as evidence.

Wildcards

Use `*` for any sequence of characters and `?` for a single character. For example, `fox*` matches "fox", "foxes", "foxy".

Export

Click **Export Excel** to download the concordance results as an Excel file.

5. Frequency Analysis

1. Select a corpus.
2. Click the **Analyze** sub-tab.
3. Click the **Frequency** tab.
4. Choose the unit: word, lemma, or POS.
5. Set a minimum frequency if desired.
6. The table shows: item, frequency, per-million, and percent.

The STTR (Standardized Type-Token Ratio) is displayed at the top. STTR is computed over 1000-token chunks and is the comparably valid default for cross-corpus comparison. Raw TTR is sample-size-sensitive and should not be used for comparison.

6. Collocation Analysis

1. Select a corpus.
2. Click **Analyze**, then **Collocation**.
3. Type a node word (for example, "research").
4. Choose the level: word or lemma.
5. Set the window size (default: 5 tokens on each side).
6. Set the minimum frequency threshold.
7. Click **Compute**.

The results table shows all 7 collocation measures for each collocate:

- bullet **MI** (Mutual Information, Church and Hanks 1990)
- bullet **T-score**
- bullet **Log-likelihood** (Dunning 1993)
- bullet **Dice**
- bullet **LogDice** (Rychly 2008)
- bullet **Chi-square**
- bullet **Delta P** (Gries 2013, directional: both directions shown)

The window size and minimum frequency are always displayed alongside the results. A collocation measure without a stated window size is not reproducible.

7. Keyness Analysis

Keyness compares your target corpus against a reference corpus to find words that are significantly more or less frequent in the target.

1. Create two corpora: one target and one reference.
2. Select the target corpus.
3. Click **Analyze**, then **Keyness**.
4. Select the reference corpus from the dropdown.
5. Set the minimum frequency.
6. Click **Compute**.

The results show both **significance tests** (log-likelihood, chi-square) and **effect-size measures** (Log Ratio, %DIFF, Simple Maths, Odds Ratio) side by side. This is a load-bearing design principle: a

"key" word is never reported as important on frequency-of-occurrence-in-a-huge-corpus grounds alone.

Positive keywords are over-represented in the target corpus. Negative keywords are under-represented.

Methods Section PDF

Click **Methods PDF** to auto-draft a methodology paragraph citing the exact tools, versions, and formulas used. You can paste this directly into your manuscript.

8. Dispersion Analysis

1. Select a corpus.
2. Click **Analyze**, then **Dispersion**.
3. Type a term.
4. Click **Compute**.

Results show two dispersion statistics:

- bullet** **Juilland's D:** ranges from 0 (maximally concentrated) to 1 (perfectly even).
- bullet** **Gries' DP:** ranges from 0 (perfectly even) to $(n-1)/n$ (maximally concentrated), where n is the number of corpus parts.

A bar chart shows the term's frequency per document.

9. N-grams and Lexical Bundles

1. Select a corpus.
2. Click **Analyze**, then **N-grams**.
3. Choose n (2 to 10).
4. Set minimum frequency and minimum range.

The minimum range is the number of distinct documents the n -gram must appear in. This is the standard frequency-and-range criterion from Biber et al. (1999): a lexical bundle requires both a minimum frequency per million words AND a minimum number of distinct texts. Raw frequency alone is not enough to distinguish genuine bundles from single-text artifacts.

10. POS, Grammar, and Dependency Analysis

POS Analysis

Click **Analyze**, then **POS**. Choose n=1 for the POS distribution, or n=2/3/4/5 for POS n-grams (for example, "DET NOUN" is a common bigram).

Grammar Analysis

Click **Analyze**, then **Grammar**. Select which patterns to detect:

- bullet **passive_voice**: AUX "be" or "get" + VERB with aux:pass dependency
- bullet **modal**: modal verbs (can, could, may, might, must, shall, should, will, would)
- bullet **negation**: "not" or "n't" as neg dependency
- bullet **relative_clause**: acl:relc dependency
- bullet **complex_np**: NOUN with 2 or more modifiers
- bullet **tense**: past, present, or future (from morphological features)

These detectors are dependency-parse-driven, not regex over surface text, so they generalize across genres.

Dependency Analysis

Click **Analyze**, then **Dependency**. Choose a relation type (nsubj, obj, iobj, obl, etc.) to see the most common governor-dependent pairs.

11. Discourse and Metadiscourse Analysis

Click **Analyze**, then **Discourse**. This runs Hyland's (2005) metadiscourse taxonomy across your corpus:

Interactive categories (how the writer organizes the text):

- bullet Transitions (however, therefore, moreover)
- bullet Frame markers (first, in conclusion, to summarize)
- bullet Endophoric markers (see figure, as noted above)
- bullet Evidentials (according to, cited in)
- bullet Code glosses (namely, in other words, e.g.)

Interactional categories (how the writer involves the reader):

- bullet Hedges (perhaps, may, appear to)
- bullet Boosters (clearly, undoubtedly, in fact)
- bullet Attitude markers (surprisingly, importantly)
- bullet Self-mentions (I, we, our)

- bullet Engagement markers (consider, note that, you)

Each result includes example sentences with stable evidence IDs.

12. Vocabulary Profiling

Click **Analyze**, then **Vocabulary**. This profiles your corpus into frequency bands:

- bullet **K1**: top 200 most frequent words (from the bundled open wordlist)
- bullet **K2-K9**: words in frequency bands 2 through 9 (currently approximated)
- bullet **AWL**: Academic Word List (starter subset of Coxhead 2000)
- bullet **Off-list**: words not in any of the above

The tool also reports rare words (frequency of 1 or less) and academic words found in the corpus.

13. Sentiment Analysis

Click **Analyze**, then **Sentiment**. Each sentence gets a score from -1 (very negative) to +1 (very positive). The results show:

- bullet Positive / negative / neutral sentence counts
- bullet Average score across the corpus
- bullet A per-sentence timeline (colored bars: green = positive, red = negative, grey = neutral)

The sentiment analysis is lexicon-based. Phase 7+ will swap in VADER or a transformers model behind the same interface.

14. Metaphor Detection

Click **Analyze**, then **Metaphor**. This produces metaphor candidates: verbs with abstract subjects that may indicate personification or metaphor.

Important: These are candidates only. The LLM triages them via MIPVU decision steps, and a human must verify before any candidate counts as a confirmed metaphor in any export or statistic. This verification gate is load-bearing for validity. Current evidence shows LLMs alone under-perform supervised detectors and especially struggle to filter literal false positives.

Each candidate shows:

- bullet The word and its lemma
- bullet The subject it co-occurs with
- bullet The full sentence
- bullet A stable evidence ID

15. Arabic Analysis

Click the **Arabic** group in the Text Suite ribbon tab. The Arabic workbench has 9 tools:

1. **Morphology**: full analysis with root, pattern, lemma, POS, stem, Buckwalter transliteration, number (singular/dual/plural), gender (masculine/feminine), and broken-plural flag.
2. **Roots**: extracts trilateral roots (for example, all words sharing the root k.t.b: kitab, maktaba, katib, yaktub).
3. **Clitics**: segments clitics into surface + stem + POS.
4. **Buckwalter**: transliterates Arabic script to ASCII Latin.
5. **Dediacritize**: removes harakat (diacritics).
6. **Normalize**: unifies alef variants, teh marbuta, alef maksura.
7. **Dialect ID**: identifies MSA, Egyptian, Gulf, or Levantine. With `include_cities=true`, also returns city-level scores (Beirut, Cairo, Doha, Rabat, Tunis).
8. **Register**: detects Classical, MSA, or Dialectal.
9. **Translate**: looks up Arabic-to-English translation equivalents.

All Arabic text in the UI uses right-to-left (RTL) layout with the Arabic font stack.

16. Image Ingestion and Vision Analysis

1. Click the **Vision** tab in the ribbon.
2. Click **Manage**.
3. Create an image set within your corpus.
4. Drag and drop images (JPG, PNG, TIFF, WebP, BMP, GIF).
5. Optionally add captions (one per image, newline-separated).
6. Select an image, then click **Analyse** to see:

bullet Colour analysis: dominant colours, warm/cold balance, brightness, contrast, saturation, colour symbolism notes

bullet Composition analysis: information value (left/right = Given/New, top/bottom = Ideal/Real, centre/margin), salience centre, rule-of-thirds intersections, visual balance, framing balance

bullet OCR text (if Tesseract is installed)

17. Visual Grammar (Kress and van Leeuwen)

Select an image, then click **Visual Grammar**. This analyses the image against Kress and van Leeuwen's (2006) three meaning-metafunctions:

- bullet **Representational:** what the image depicts (narrative vs conceptual processes)
- bullet **Interactive:** the relationship the image constructs between viewer and represented (modality, colour symbolism)
- bullet **Compositional:** how elements are integrated (information value, salience, framing)

Every claim is framework-attributed and phrased as a hypothesis: "Under a Kress and van Leeuwen reading, X may indicate Y." Claims are color-coded by metafunction and include evidence citations.

18. Multimodal Image-Text Alignment

Select an image, then click **Align**. Type or paste the co-occurring text (a caption, article body, etc.) and click **Align**.

The alignment engine:

1. Extracts image regions (3x3 grid, 9 regions)
2. Extracts text spans (colour terms, positional terms, keywords)
3. Matches regions to spans using heuristic similarity (colour-term matching + positional hints)
4. Returns each alignment with a confidence score and match reason

Phase 5 will swap in CLIP-style embeddings behind the same interface.

Cross-modal relations (reinforcement, complementarity, silence) are detected and displayed.

19. Multimodal Discourse Analysis

Select an image, then click **Discourse**. Choose from 8 analyses:

1. **Social Semiotic** (Kress and van Leeuwen 2006)
2. **CDA** with 4 selectable frameworks: Fairclough, van Dijk, Wodak, Machin and Mayr
3. **Persuasion** (Aristotle's ethos/pathos/logos + Toulmin's argument structure)
4. **Framing** (Entman's 4 functions)
5. **Narrative** (Labov's 6-stage structure)
6. **Visual Metaphor** (MIPVU-inspired, candidates only, human verification required)
7. **Emotion** (combined image colour + text lexicon)
8. **Cultural** (Barthes + Anderson, always culture-relative)

Every claim is framework-attributed and phrased as a hypothesis per the design principles. Never state ideology, bias, or power relations as settled fact.

20. The AI Assistant

Click the **AI Assistant** tab in the ribbon. The Assistant is a tool-using agent, not a chatbot.

How It Works

1. You ask a question in natural language.
2. The Assistant selects and calls the appropriate tool (search_concordance, compute_collocations, etc.).
3. The tool returns deterministic, cited evidence.
4. The Assistant writes its answer based on that evidence.
5. Every claim in the answer is clickable back to its source.

Grounded vs. Ungrounded

If the Assistant calls a tool, the answer is marked **grounded** (green badge). If no tool was called (for example, you asked a general knowledge question), the answer is marked **ungrounded** (orange badge). This is the load-bearing implementation of the grounded-AI principle: the UI never silently presents an ungrounded answer as equal-weight fact.

Example Questions

- bullet "What are the top 10 most frequent words in this corpus?"
- bullet "Find all occurrences of 'research' and show me their contexts."
- bullet "What are the strongest collocates of 'dog' within 5 tokens?"
- bullet "Compare this corpus against the reference. What are the top keywords?"
- bullet "How evenly is 'the' distributed across the documents?"
- bullet "What hedges does this author use?"
- bullet "Analyse this image using Kress and van Leeuwen."

21. Exporting Results

Excel Export

Available for: concordance, frequency, collocations, keyness. Click the **Export Excel** button on any analysis tab.

Methods Section PDF

Available from the Keyness tab. Auto-drafts a methodology paragraph naming the exact tools, versions, and formulas used. Includes citations for the statistical measures (Church and Hanks 1990, Dunning 1993, Rychly 2008, Gries 2013, Hardie 2014, etc.).

22. Saved Searches, Bookmarks, and Favorites

Saved Searches

Save any search query with its parameters for re-use. Go to the Manage tab, select a project, and use the saved-searches API.

Bookmarks

Bookmark specific concordance lines, statistics, or image regions. Each bookmark can have a label and a note.

Favorites

Pin frequently-used corpora, searches, or conversations for quick access.

23. Project Sharing and Collaboration

Share a Project

1. Go to Settings or use the API.
2. Mark a project as shared (private or public).
3. A share token is generated for public access.

Save and Sync

Phase 6 implements save-and-sync, not real-time CRDT co-editing. Each sync push/pull is logged in an audit trail. This is a deliberate scope decision: CRDT infrastructure is a significant complexity increase that most research teams do not need.

24. Arabic-Specific Features Deep Dive

Root Extraction (al-jizr)

Arabic morphology is root-and-pattern based. Most Arabic words derive from a trilateral (three-consonant) root by applying a morphological pattern that interleaves vowels and affixes among the root consonants.

For example, the root k.t.b (write) produces:

- bullet kitab (book) via the pattern 1i2a3
- bullet maktaba (library) via the pattern ma12a3a
- bullet yaktub (he writes) via the pattern ya12u3
- bullet kuttab (writers) via the pattern 1u22a3

CorpusMind extracts the root and pattern for each token using CAMEL Tools' morphology analyzer (calima-msa-r13 database).

Dialect Identification

The full CAMEL DIDModel6 distinguishes 6 city dialects:

- bullet Beirut (Levantine)
- bullet Cairo (Egyptian)
- bullet Doha (Gulf)
- bullet MSA (Modern Standard Arabic)
- bullet Rabat (Maghrebi)
- bullet Tunis (Maghrebi)

City scores are aggregated into the four standard CorpusMind dialect buckets: msa, egy, glf, lev. Maghrebi dialects currently fall under the MSA bucket.

Broken Plurals

A broken plural (jama taksir) is a plural noun whose pattern differs from the regular sound-plural patterns. CorpusMind flags broken plurals automatically by checking whether the pattern matches the sound-plural templates (1u2una / 1u2iina for masculine, 1a2a3at for feminine).

25. Bilingual Corpus Tools

Sentence Alignment

Align two parallel corpora (Arabic and English) at the sentence level using the Gale-Church (1993) length-based algorithm. Each aligned pair includes a confidence score.

Parallel Concordance

Search the Arabic side and get each hit paired with its English translation (per the sentence alignment). Results show KWIC format side by side.

Translation Equivalents

Look up Arabic-to-English or English-to-Arabic translation equivalents. Phase 6 uses a starter dictionary of 50+ high-frequency words. Phase 7+ will integrate a proper bilingual word-alignment model.

26. Facial Analysis (Opt-In)

Facial analysis is **OFF by default** per the ethical guardrails. It must never be silently enabled.

To Enable

Set the environment variable:

```
export CORPUSMIND_FACIAL_ANALYSIS_ENABLED=1
```

What It Does

- bullet Detects faces using OpenCV Haar cascades
- bullet Produces descriptive visual cues: facial expression, head direction
- bullet Each result includes an ethics notice

What It Does NOT Do

- bullet It does NOT perform identity recognition or re-identification of real individuals
 - bullet It does NOT estimate age group or gender presentation in Phase 6 (legally sensitive, unreliable without a trained model)
 - bullet It does NOT store biometric templates
-

27. At-Rest Encryption

To Enable

Generate a 32-byte encryption key:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Set it as an environment variable:

```
export CORPUSMIND_ENCRYPTION_KEY="your-64-character-hex-key-here"
```


What Gets Encrypted

Image files stored on disk are encrypted with AES-256-GCM. The encryption key is read from the environment variable and is never stored on disk.

Warning

If you lose the encryption key, all encrypted data becomes unrecoverable. Store the key somewhere safe (a password manager, a hardware security module, etc.).

Check the encryption status at any time in Settings or via the API endpoint `/api/v1/encryption/status`.

28. Self-Hosted Lab Deployment

For a research lab that wants one always-on engine instance shared by multiple group members:

Quick Start

```
cd CorpusMind docker compose -f infra/docker-compose.yml up -d
```

With TLS

```
docker compose -f infra/docker-compose.yml --profile tls up -d
```

Place your TLS certificates in `infra/certs/` (fullchain.pem and privkey.pem).

Configuration

Edit `infra/.env.example` and copy to `engine/.env`:

- bullet `CORPUSMIND_CLOUD_DISABLED_HARD=true` (blocks all cloud routing on shared infrastructure)
- bullet `CORPUSMIND_ENCRYPTION_KEY` (optional, enables at-rest encryption)
- bullet `CORPUSMIND_CORS_ORIGINS` (add your lab's PWA origin)

Members point their PWA at `https://your-lab-server:8765`.

29. Accessibility Features

CorpusMind targets WCAG 2.1 AA compliance:

- bullet **Visible focus indicators** for keyboard navigation (green outline on focused elements)
- bullet **Skip-to-content link** at the top of every page (press Tab to reveal)
- bullet **Screen-reader-only text** for icons and decorative elements
- bullet **High contrast mode** support (respects OS preference)

- bullet **Reduced motion** support (respects OS preference)
- bullet **44px minimum touch targets** on all interactive elements
- bullet **Full RTL mirroring** for Arabic (menus, ribbon, alignment, not just text direction)
- bullet **Semantic HTML** with proper ARIA roles (main, contentinfo)
- bullet **Keyboard shortcuts**: Ctrl/Cmd+K opens the command palette

RTL Mode

Go to View > Direction > RTL, or use the command palette. The entire UI flips to right-to-left layout. All Arabic text uses the Arabic font stack (Noto Naskh Arabic, Amiri, Scheherazade New).

30. Troubleshooting

Engine will not start

```
lsof -i :8765
```

If another process is using port 8765, either kill it or set a different port:

```
export CORPUSMIND_PORT=8766 corpusmind-engine
```

Ollama shows "offline" in Settings

Make sure `ollama serve` is running in a separate terminal. Verify:

```
curl http://127.0.0.1:11434/api/tags
```

You should see JSON with your downloaded models.

Arabic analysis returns errors

Make sure you have installed the CAMEL Tools data:

```
camel_data -i morphology-db-msa-r13 camel_data -i dialectid-model6
```

Verify the database exists:

```
ls ~/.camel_tools/data/morphology_db/calima-msa-r13/morphology.db
```

OCR returns "none" engine

Install Tesseract:

```
brew install tesseract pip install pytesseract
```

For Arabic OCR, also install the Arabic language pack:

```
brew install tesseract-lang
```

Desktop app (Tauri) fails to build

Make sure you have the system dependencies:

On macOS: `xcode-select --install`

On Linux: `sudo apt install libwebkit2gtk-4.1-dev
libayatana-appindicator3-dev librsvg2-dev`

On Windows: install the Visual Studio C++ Build Tools.

Upload fails with "unsupported format"

Check the file extension. Supported formats: .txt, .md, .docx, .pdf, .html, .htm, .xml, .csv for text; .jpg, .jpeg, .png, .tif, .tiff, .webp, .bmp, .gif for images.

AI Assistant answers are empty or time out

Make sure you have pulled a model:

```
ollama list
```

If the list is empty, pull a model:

```
ollama pull llama3.2:3b
```

If you have limited RAM, use the 3B model. If you have 16+ GB RAM, the 8B model gives better answers.

Database errors after upgrading

Delete the old database and restart:

```
rm ~/.corpusmind/corpusmind.db corpusmind-engine
```

Note: this deletes all projects and corpora. Export anything you want to keep before deleting.

Getting Help

- bullet **GitHub Issues:** <https://github.com/waleedmandour/CorpusMind/issues>
- bullet **User Guide:**
https://github.com/waleedmandour/CorpusMind/blob/main/docs/USER_GUIDE.md
- bullet **Changelog:** <https://github.com/waleedmandour/CorpusMind/blob/main/CHANGELOG.md>
- bullet **Methodology Reference:**
<https://github.com/waleedmandour/CorpusMind/blob/main/docs/METHODOLOGY.md>
- bullet **Architecture:**
<https://github.com/waleedmandour/CorpusMind/blob/main/docs/ARCHITECTURE.md>